# Data Warehouse Project

*Data Warehousing and Visualization Project Report*

Alessandro Sidero 276592

Università della Calabria
DeMaCs - Department of Mathematics and Computer Science
Master in Artificial Intelligence and Computer Science
Data Warehouse and Visualization

A.Y. 2025/2026

# Contents

## List of Figures

## List of Tables

## Introduction

This report presents the design and implementation of a data warehouse for 2022 US domestic flight departures, built on the public *2022 US Airlines Domestic Departure Data* dataset [1]. The source contains roughly 6.95 million departure events, combining flight operations, carrier information, aircraft characteristics, airport metadata, and hourly weather observations at origin airports.

The analytical goal is to study delays and cancellations. The warehouse connects each departure with time, geography, carrier, aircraft, and weather context, making it possible to locate where disruptions concentrate, identify when they become more likely, and examine which operational or environmental factors help explain them.

The project follows the required data warehouse workflow: source analysis, reconciled database design, data quality assessment, cleaning, dimensional modeling, and ETL implementation. Data quality is treated as a central part of the method, not a preliminary formality. Cleaning choices are therefore justified and logged, particularly when they affect missing weather observations, physical consistency rules, or operational outliers.

LLM support was limited to documentation and review tasks. The main local model was `llama3.2:3b` through Ollama; the notebooks also support optional API-based calls through OpenRouter, but these are treated as review assistance only. Prompts were built from schemas, summary statistics, validation outputs, and JSON-like metadata; raw datasets were not passed to the model. The LLM was used for three specific tasks: interpreting DQA scorecards, interpreting missingness summaries, and translating selected outlier results into operational explanations. No model output was accepted without checking against deterministic notebook results, domain rules, or Phase 1 design artifacts. Where LLM assistance is referenced, the corresponding system and user prompts are shown in the relevant section.

The implementation uses three tools with distinct roles:

- **Python (Jupyter Notebooks)**: data quality assessment, profiling, and ETL implementation.
- **Local and API-based LLMs (Ollama / OpenRouter)**: controlled support for selected interpretation and documentation tasks.
- **PostgreSQL**: hosts both the Reconciled Relational Schema and the final dimensional Data Warehouse.

## Domain Analysis

A flight departure is not an isolated event. It can be affected by the departure airport, the time of day, the operating carrier, the assigned aircraft, and the local weather at departure. Delays also propagate: a late-arriving aircraft may force the next rotations on the same tail number into progressively worse conditions. The project therefore treats the departure event as the unit of analysis and enriches it with surrounding operational context.

Three questions drive the analysis. Where and when do disruptions concentrate across the US domestic network? Do ground operations, carrier behavior, aircraft age, or manufacturer groups show distinct delay and taxi-out patterns? And under which combinations of wind, visibility, temperature, and cloud cover does the risk of cancellation become elevated?

These translate into three operational objectives:

- Which airports, months, and departure time bands concentrate the highest delay and cancellation pressure?
- Is there a measurable relationship between taxi-out duration, carrier behavior, aircraft age, manufacturer group, and delay rate?
- Which weather contexts are associated with higher disruption risk, without over-interpreting isolated sensor minima?

The expected output is a set of findings that separates operational inefficiencies from weather-driven disruptions. Aggregate statistics often hide this distinction, so the dimensional model is designed to make it visible.

## Dataset Description

The dataset used in this project is the 2022 US Airlines Domestic Departure Data [1], a public Kaggle dataset with approximately 6.95 million departure events. The original files are stored under `data/1-original dataset/`. The source is not already organized as a warehouse: the main table is a large denormalized file, while smaller CSV files provide code descriptions and reference metadata.

`CompleteData.csv` is the primary relation. It contains 40 attributes across approximately 6.95 million flight records and combines operational data, aircraft metadata, airport geography, and hourly weather observations. The auxiliary files help interpret encoded values and descriptive entities:

- `Carriers.csv`, describing airline carrier codes;

- `Stations.csv`, containing airport and Mesonet station metadata;
- `Cancellation.csv`, mapping cancellation status codes to semantic categories;
- `ActiveWeather.csv`, describing encoded weather conditions.

For selected attributes whose meaning extends beyond the dataset itself, descriptions were checked against aviation and meteorological references. Table 1 lists the main reference groups; it is not a second data dictionary, but explains why certain column descriptions draw on external aviation or weather documentation rather than the Kaggle source alone.

| **Attribute group** | **Examples** | **Reference** |
|---|---|---|
| Operational flight times | `DEP_TIME`, `TAXI_OUT`, `AIR_TIME`, `DEP_DELAY`, `DISTANCE` | BTS airline on-time performance glossary and table documentation [2], [3]. |
| Cancellation categories | `CANCELLED`, `CANCELLATION_REASON` | BTS on-time reporting directive, which defines carrier, weather, NAS, and security categories [4]. |
| Aircraft registration | `TAIL_NUM` | FAA Aircraft Registry and N-number inquiry documentation [5]. |
| Weather observations | `WIND_SPD`, `WIND_GUST`, `VISIBILITY`, `TEMPERATURE`, `ALTIMETER` | National Weather Service ASOS material and Iowa Environmental Mesonet ASOS/METAR archive documentation [6], [7]. |

Table 1: External references used to validate selected source attribute descriptions.

Profiling across the full 6.95 million records established the assumptions underlying the data model. The central one concerns weather observations. Weather attributes were verified to depend uniquely on the combination of station, date, and departure hour:

$$(\texttt{MESONET\_STATION}, \texttt{FL\_DATE}, \texttt{DEP\_HOUR})$$

The dependency was validated across 1,133,884 distinct groups with zero violations, confirming that weather observations are uniquely determined by station, date, and departure hour.

A bijective relationship between `ORIGIN` airport codes and `MESONET_STATION` identifiers was verified across all 375 origin airports: each departure airport maps to exactly one weather station and vice versa.

The same uniqueness property does not hold for destination airports. Profiling identified 311 violations for the dependency:

$$\texttt{DEST} \rightarrow \texttt{MESONET\_STATION}$$

confirming that weather observations are exclusively tied to the departure airport. In practical terms, the warehouse can safely attach weather context to the origin airport and departure hour, but it should not infer destination weather from the same station field.

Flight cancellations in this dataset are assigned to one of four operational categories, summarized in Table 2.

| STATUS | CANCELLATION_REASON |
|---|---|
| 0 | Not Cancelled |
| 1 | Carrier Cancellation |
| 2 | Weather Cancellation |
| 3 | National Air System Cancellation |
| 4 | Security Cancellation |

Table 2: Semantic mapping of flight cancellation status codes to operational categories.

Table 2 is used throughout the project to interpret cancellation status values. In the final warehouse, these categories are represented inside `DIM_JUNK` together with the active weather description.

A dedicated consistency check was run on the 147,830 cancelled flights. Despite their status, all records preserved coherent temporal information. `DEP_TIME` matched `FL_DATE` and `DEP_HOUR` in 100% of cases, and `DEP_DELAY` was never null.

810 of these flights had non-zero `TAXI_OUT`, which suggests that ground procedures had already started before cancellation was registered. No cancelled flight had positive `AIR_TIME`, so the anomaly is limited to pre-departure operations.

Cancelled flights therefore retain useful operational and environmental information and are carried through the cleaning and ETL stages without removal.

## Reconciled Database Design

The raw CSV files follow a flat, denormalized export format. The re-engineering step decomposes this structure into relational entities, resolves export-related type issues, and introduces surrogate keys for the transactional tables. At this stage the objective is a stable staging layer that preserves the original granularity, not yet a dimensional structure.

Surrogate keys (`Flight_ID` and `Obs_ID`) were introduced for the two high-volume transactional tables. Reference entities keep their natural keys in the staging layer, which simplifies lookup operations during loading. The `Weather_Observation_ID` foreign key in the Flight table is resolved after loading through a join on the natural matching conditions.

The schema organizes the domain into seven tables, as shown in Figure 1. `Flight` and `Weather_Observation` contain the transactional data. `Aircraft`, `Carrier`, and `Station` store descriptive metadata. `Active_Weather` and `Cancellation` are compact lookup tables for domain status codes.


Figure 1: Schema of the reconciled database.

Figure 1 shows the intermediate relational layer used before the warehouse. Its purpose is to separate repeated descriptive entities from the large flight and weather tables while preserving all information needed by the ETL.

Foreign key and unique constraints are intentionally absent from the staging DDL. Enforcing them during bulk ingestion would add validation overhead and dependency-ordering issues. Referential integrity is instead checked in the Python/Pandas pipeline before data is promoted to the warehouse schema.

Several columns that are conceptually boolean or integer-coded are stored as `NUMERIC(3,1)`, including `Low_Level_Cloud` and `Active_Weather_Status`. This comes from Pandas type inference: `NaN` values force an upcast to floating-point during CSV parsing. These inconsistencies are tolerated in staging and corrected during transformation.

## Reconciled Database Implementation

The physical schema is implemented in PostgreSQL through `phase 1 - design/2-Reconciled Database.sql`. Since strict constraints are relaxed in staging, the implementation focuses on read performance for the downstream ETL. Indexes are created on the columns most often used in filters and joins: `FL_Date`, `Origin`, `Dest`, `MKT_Unique_Carrier`, and `Cancelled`.

The ingestion sequence follows the logical dependencies without relying on database-level enforcement. Reference tables are loaded first, followed by the high-volume transactional tables. Once `Flight` and `Weather_Observation` are available, a post-load update populates `Weather_Observation_ID` through a join on:

$$\texttt{Origin} = \texttt{Origin\_Airport}, \quad \texttt{FL\_Date} = \texttt{Obs\_Date}, \quad \texttt{Dep\_Hour} = \texttt{Obs\_Hour}$$

At this stage, the schema absorbs the raw data without loss, preserving all artifacts and type inconsistencies that will be addressed in the cleaning phase. The loaded tables serve as input to the data quality assessment pipeline.

## Data Quality Assessment

The Data Quality Assessment (DQA) was conducted before transformation. It follows the ISO 25012 dimensions: Completeness, Uniqueness, Validity, Consistency, and Timeliness. Each dimension was evaluated in `phase 2 - etl/1-Data Quality Assessment.ipynb`. The resulting scorecards guided the cleaning decisions. Figure 2 summarizes the pre-cleaning scores.

The DQA figures used in the report are generated from the scorecard CSV files through `phase 4 - report/DQA Figure Generation.ipynb`. In Figures 2 and 3, the color gradient maps lower quality scores to red and higher scores to green; blank cells indicate dimensions that are not applicable to a table.


Figure 2: DQA scorecard: pre-cleaning scores per table and quality dimension.

Figure 2 makes clear that the main issue is not the Flight table itself, which already scores highly, but the Weather_Observation table. Its low uniqueness and consistency scores explain why weather cleaning becomes one of the central steps of the project.

LLM assistance was used to translate the most critical DQA scorecard into an interpretable data-quality risk:

**System prompt**

```
You are a Data Quality Auditor for aviation datasets.
Review scorecards and identify critical risks for downstream analytics.
```

**User prompt**

```
Review the DQA scorecard for the Weather_Observations table.
Explain the business risk of low uniqueness and consistency scores.
```

The response correctly identified the Weather_Observation table as the main DQA risk: duplicated hourly observations and inconsistent meteorological values could bias delay analysis. The remediation strategy, however, was defined entirely by deterministic notebook rules.

### Completeness and Missingness

Reference entities (Carrier, Aircraft, Station) exhibited near-perfect completeness. The single exception was a missing `range` value in the Aircraft table (0.02% of records), treated as negligible.

The Weather_Observation table showed a uniform missingness pattern. The relevant meteorological variables had 32,789 missing rows, equal to about 0.47% of the denormalized weather records. Since the count was identical across variables, missingness was row-level rather than column-specific.

A co-occurrence analysis showed that the 32,789 affected flight records mapped to weather rows where all selected weather fields were missing. Only 2,497 of those flights were cancellations, equal to 7.62% of the affected subset and below the 10% safety threshold used for the dropping decision. The mechanism was also investigated statistically. The Jaro-Winkler correlation and a logistic regression for `wind_gust` produced pseudo-$R^2 = 0.031$, while the chi-squared test against `obs_hour` was significant ($p = 0.0$). This suggested MAR (Missing At Random) rather than MCAR. Given the low volume and the limited cancellation exposure within the affected subset, the final decision was to drop these rows after programmatic validation (see Section 7.2).

LLM assistance was used only to frame the interpretation of this missingness pattern. The prompt was intentionally based on aggregate metadata rather than raw rows:

**System prompt**

```
You are a Data Scientist specializing in aviation databases.
Interpret statistical missingness in tables like Flights, Weather,
Stations, or Aircraft. Explain if it is MCAR, MAR, or MNAR based on
real-world aviation logic. Recommend a DW strategy.
```

**User prompt**

```
Review the missingness statistics for the Weather_Observations table.
Provide: classification, operational context, and ETL handling strategy.
```

The model suggested checking the temporal structure before choosing between imputation and deletion. This was useful as a checklist, but the final decision was based on the co-occurrence and temporal-gap checks described above.

Validity-range checks additionally identified residual out-of-range meteorological values. Attributes not used in the warehouse, such as `rel_humidity` and `altimeter`, were allowed to remain nullable in the cleaned reconciled layer. By contrast, residual `VISIBILITY` nulls were imputed before the ETL using the median visibility of the same origin airport, with the global median as fallback. This prevents a nullable analytical measure from being propagated into the final fact table.

### Uniqueness and Duplication

The main redundancy was structural. Since the raw data is denormalized at flight-event level, each hourly weather observation is repeated once for every flight departing from the same origin in that hour. The Weather_Observation staging table contained 6,954,636 rows instead of 1,133,884 unique hourly readings before cleaning. After removing the weather rows with all selected meteorological fields missing, the cleaned table contained 1,121,382 hourly observations. This required exact deduplication on `(origin_airport, obs_date, obs_hour)`.

A separate fuzzy deduplication analysis was run on the Station table using Jaro-Winkler similarity with blocking on `airport_state_code`. It found near-duplicate names such as Chicago O'Hare (ORD) vs. Chicago Midway (MDW) and Phoenix Sky Harbor (PHX) vs. Phoenix-Mesa (AZA). These were confirmed as distinct airports and retained.

### Consistency and Validity

Inter-column consistency checks exposed two classes of meteorological anomalies. First, over 800,000 records had sustained wind speed greater than wind gust, which is physically inconsistent. Second, some records had dew point above ambient temperature, which is not valid under standard atmospheric conditions.

Structural validity issues were also recorded. Some conceptually boolean columns were represented as `NUMERIC(3,1)`, as already observed in the staging schema. Another inconsistency involved `cloud_cover`: some rows had value 0 even when cloud-layer flags were active or `lowest_cloud_layer` was below 10,000 ft.

### Outlier Assessment

Outlier analysis was used as a decision-support step, not as an automatic deletion rule. IQR, Modified Z-score, and Isolation Forest were applied only to variables that can support the analytical goals: delay, taxi-out, air-time, wind gust, and selected weather context measures. Geographic attributes such as latitude, longitude, and elevation were excluded from the final interpretation because extreme values in those fields mostly describe the real geography of US airports. Route distance was also excluded from the final outlier discussion, since long distances are structural route properties rather than anomalous operational events.

The most important result concerns `dep_delay`. This variable has a long right tail: IQR with $k = 3.0$ flagged 4,067 records, while Modified Z-score flagged 7,661 records. These values are not necessarily errors. Very high delays can represent real operational disruptions, such as severe weather, aircraft rotation problems, or

cascading airport congestion. For this reason, `dep_delay` was kept uncapped in the warehouse.

`taxi_out` and `air_time` were interpreted differently. Their outliers are less useful as extreme analytical events because both measures are constrained by physical and operational plausibility. IQR flagged 2,942 records for `taxi_out` and 2,600 for `air_time`. These variables were therefore selected for controlled winsorization during cleaning, while preserving the flight records themselves.

Weather extremes were treated as contextual information rather than direct data errors. `wind_gust` remains useful for outlier inspection because high gust values can represent disruptive weather. `visibility`, instead, is interpreted mainly through averages or threshold rates rather than as a simple IQR outlier: many readings are at or near zero, and using the minimum alone can overstate the importance of a single sensor value. The multivariate weather check is therefore used to identify unusual weather combinations rather than rows to delete.

The final decision is therefore selective: preserve meaningful operational extremes, cap only measures with physical plausibility limits, and avoid reporting outliers that do not support the project questions.

LLM assistance was used to check whether extreme values should be interpreted as errors or valid operational events:

**System prompt**

```
You are an Aviation Operations Analyst.
Translate quantitative outliers into operational realities such as
ATC delays, storms, sensor failures, or data-entry errors.
```

**User prompt**

```
We are analyzing numeric extremes in the Flights table.
Provide likely root causes, data validity interpretation,
and impact on analytics for the selected columns.
```

The output helped distinguish operational extremes from probable data artifacts. It supported keeping `dep_delay` uncapped, while allowing `taxi_out` and `air_time` to be capped by controlled cleaning rules.

## Data Cleaning Strategy

The cleaning logic is implemented through a `CleaningPipeline` class in `phase 2 - etl/5-Cleaning Pipeline.ipynb` and `phase 2 - etl/utils/cleaning.py`. The pipeline is organized into chainable transformation methods. Each method logs row-level and column-level changes through an embedded `AuditLog`. Across the full run, the audit log recorded over 2.3 million transformations, supporting traceability and reproducibility.

### Deduplication and Referential Integrity

The Weather_Observation staging table was deduplicated on `(origin_airport, obs_date, obs_hour)` after the null-row safety check. The preliminary analysis identified 1,133,884 distinct hourly groups in the reconciled layer; after removing weather observations with all selected meteorological fields missing, the cleaned layer contained 1,121,382 hourly observations. Flight records linked to removed duplicates were then remapped to the surviving canonical observation.

Referential integrity was enforced programmatically. In `phase 2 - etl/utils/cleaning.py`, the method `fix_fk_to_null` sets broken foreign-key values to `NULL`; in `phase 2 - etl/5-Cleaning Pipeline.ipynb`, this method is called for `tail_num` against the cleaned Aircraft table. This preserves flight records while marking unresolved aircraft references clearly.

### Null Handling

Before dropping weather rows, the pipeline ran a temporal gap check grouped by `origin_airport`. The goal was to verify that removing the weather rows with all selected meteorological fields missing would not create critical breaks in the hourly time series. In this dataset, the operation removed the corresponding 32,789 flight records and reduced the Flight table from 6,954,636 to 6,921,847 rows.

Records where weather values exceeded the accepted physical ranges were coerced to `NULL` as part of validity-range enforcement, handled by the general-purpose `fix_validity_range` method. Since `VISIBILITY` is a warehouse measure, its few residual nulls are then imputed with a conservative median-based rule grouped by `origin_airport`.

### Outlier Management

Winsorization at the 99th percentile was applied to `taxi_out` and `air_time` to cap physically improbable extreme values while preserving the shape of the operational

distribution.

`dep_delay` was deliberately excluded from winsorization. Multi-day groundings and cascading cancellations are not measurement errors; they are operational events that the warehouse should preserve. Capping them during ETL would remove information that cannot be recovered later.

### Business Logic Repairs

Four targeted repairs addressed the structural anomalies identified during the DQA:

**Wind gust floor.** Where `wind_gust` < `wind_spd`, the gust value was floored to the sustained speed:

$$\texttt{wind\_gust} = \max(\texttt{wind\_gust},\ \texttt{wind\_spd})$$

Swapping the two values was rejected as a strategy, since it would propagate failed sensor readings rather than correct them.

**Dew point swap.** Where `dew_point` > `temperature`, the two columns were swapped to restore thermodynamic consistency. This heuristic is appropriate only for small inversions likely caused by sensor transposition; larger deviations would require nullification.

**Cloud cover correction.** The accepted domain for `cloud_cover` is 0–4, where 0 represents clear sky. Records with `cloud_cover` = 0 but active cloud layer flags or `lowest_cloud_layer` $\leq$ 10,000 ft were corrected to `cloud_cover` = 1, the lowest non-clear category. This rule is implemented by `fix_cloud_cover_consistency` in `phase 2 - etl/utils/cleaning.py`.

**Cancelled air time.** Cancelled flights with positive `air_time` had that field set to `NULL` by `fix_cancelled_air_time` in `phase 2 - etl/utils/cleaning.py`. This is consistent with the pre-departure nature of cancellations established during the DQA.

### Validation Results

The post-cleaning DQA scorecard showed that the main warehouse-relevant quality issues were resolved for both the Weather_Observation and Flight tables. Figure 3 shows the before/after comparison.

The uniqueness score for Weather_Observation moved from 6.29% before deduplication to 100%. The low pre-cleaning score does not contradict the approximately 1.1 million distinct hourly observations: the DQA uniqueness

metric flags row instances duplicated on the business key, while the preliminary analysis counts unique weather groups. Consistency also reached 100% after the wind gust, dew point, and cloud cover repairs. Minor acceptable type warnings, such as `air_time` changing from integer to floating point, are caused by numeric transformations and nullable measure handling; they are non-fatal schema artifacts. Residual nulls in cleaned weather attributes not loaded into the warehouse are documented, while `VISIBILITY` is imputed before fact loading.


Post-cleaning scorecard.


Figure 3: Post-cleaning DQA scorecard and score improvement after the cleaning pipeline.

Figure 3 summarizes the effect of the cleaning phase: the largest gains occur for Weather_Observation uniqueness and consistency, while already stable reference tables remain unchanged. The delta view is included to avoid interpreting unchanged high scores as missed cleaning opportunities.

## Conceptual Data Warehouse Design

### Fact and Grain

The selected fact is the flight departure. The grain is one row per cleaned domestic departure event. This is appropriate because delay, taxi-out, air-time, distance, cancellation status, aircraft, carrier, airport, date, hour, and weather context are all observed at departure level.

### Attribute Tree Editing

The initial attribute tree was built from the reconciled `Flights` table, using `FLIGHT_ID` as the root because each record represents one departure event. Carrier descriptions, cancellation reasons, and weather descriptions were grafted from lookup tables. Purely operational identifiers, scheduled timestamps, airport registry codes, and unused weather details were pruned. Aircraft age was derived from `YEAR_OF_MANUFACTURE`. Airport state was retained through the city hierarchy. The edited tree in Figure 4 keeps the attributes useful for OLAP analysis and removes details that do not support the selected questions.

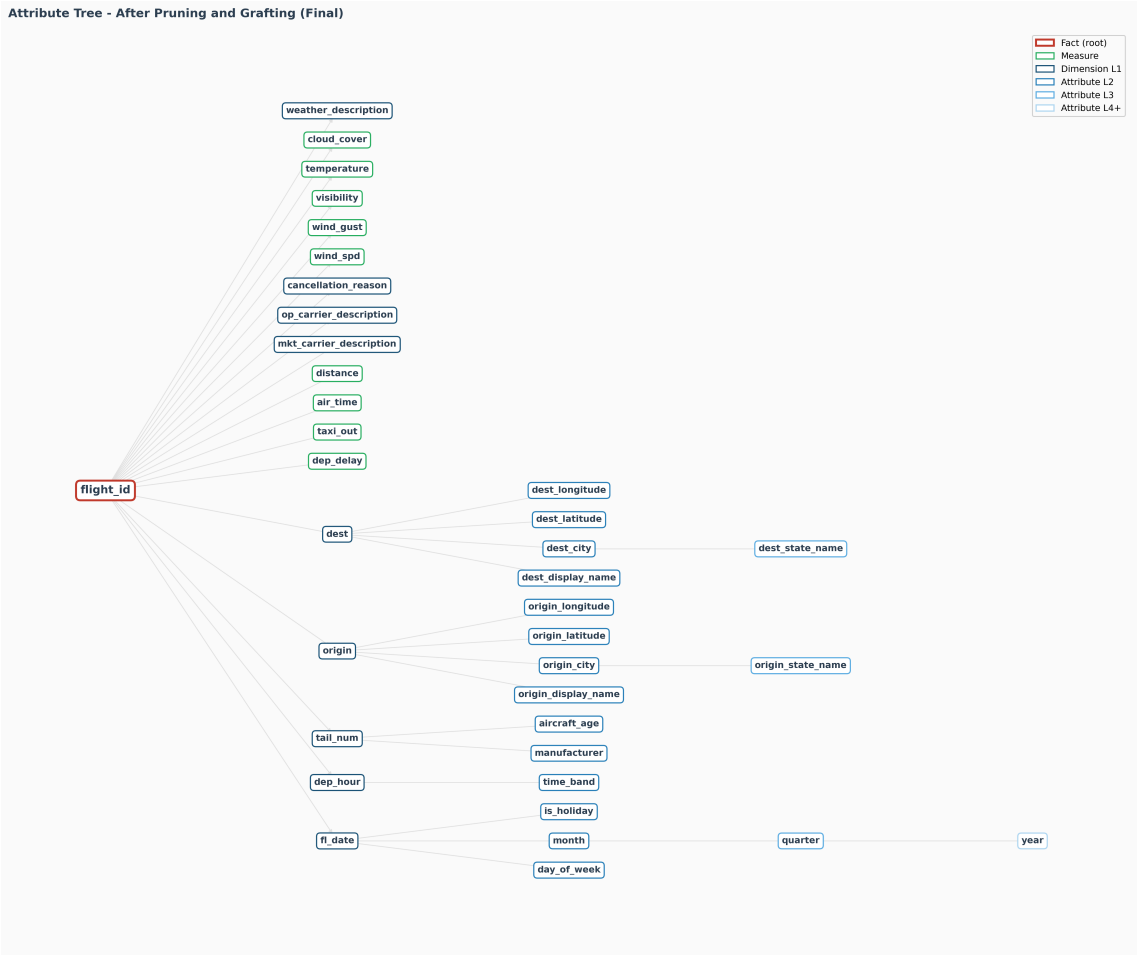


Figure 4: Edited attribute tree after pruning and grafting.

### Dimensional Model Design

The Dimensional Fact Model in Figure 5 was derived from the cleaned reconciled schema. Attributes describing the same business perspective were grouped into dimensions: flight date, departure hour, stations, carriers, aircrafts, and a junk dimension for compact categorical indicators. This keeps the fact table focused on measures and foreign keys.

Two dimensions are shared across multiple roles. The station hierarchy is used both for origin and destination analysis, while the carrier dimension is used for marketing and operating carrier perspectives. This avoids duplicating the same descriptive information and keeps roll-up paths consistent. Descriptive attributes are kept close to the dimension they explain: airport name, city, state, latitude, and longitude describe stations; manufacturer and aircraft age describe aircrafts; time band describes departure hour; cancellation reason and weather description are compact descriptors in the junk dimension.

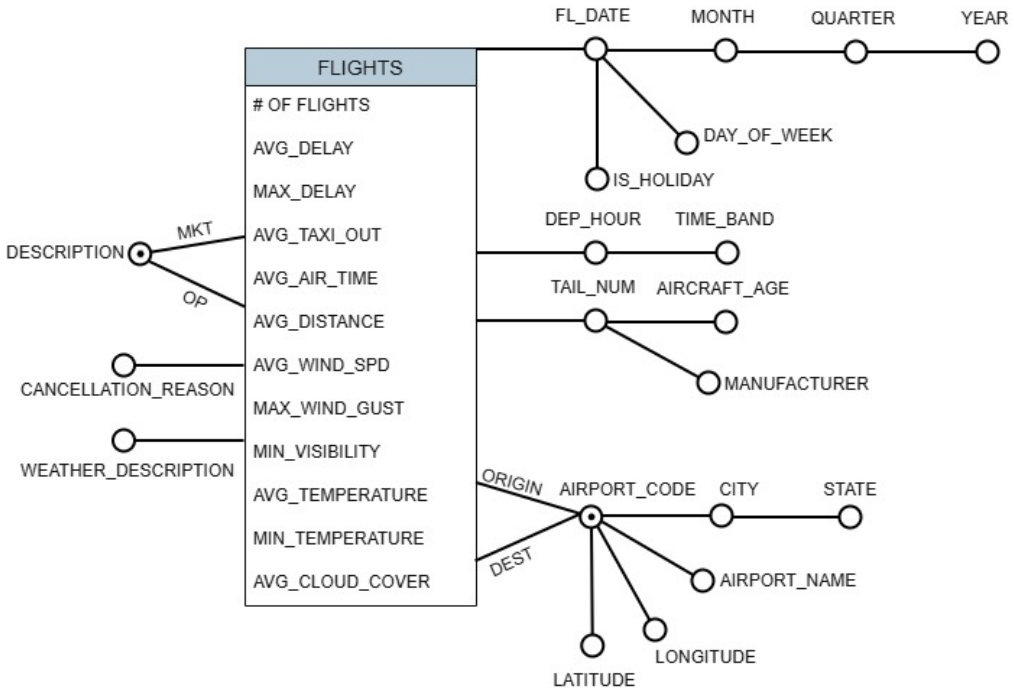


Figure 5: Dimensional Fact Model for the flight departure fact.

Figure 5 shows the selected fact, its measures, and the dimensions available for OLAP analysis. The shared hierarchies make it possible to compare origin and destination geography or marketing and operating carriers without creating separate copies of the same dimension.

### Measures

The main base measures are `DEP_DELAY`, `TAXI_OUT`, `AIR_TIME`, `DISTANCE`, `WIND_SPD`, `WIND_GUST`, `VISIBILITY`, `TEMPERATURE`, and `CLOUD_COVER`. Table 3 follows the measure glossary defined in `phase 1 - design/5-From DFM to Star Schema.md` and refined in `phase 2 - etl/7-From DFM to Star Schema.ipynb`. Additivity is reported with respect to the underlying base measure semantics: for example, delay minutes are additive, while rates and weather snapshots must be recomputed or aggregated with non-sum operators.

| Measure | Formula / Expression | Additivity | Analytical use |
|---|---|---|---|
| # OF FLIGHTS | `COUNT(*)` | Additive | Volume, map size, denominators. |
| AVG_DELAY | `AVG(DEP_DELAY)` | Additive | Delay severity by airport, carrier, route, and weather context. |
| SUM_DELAY | `SUM(DEP_DELAY)` | Additive | Total delay minutes in volume-oriented views. |
| MAX_DELAY | `MAX(DEP_DELAY)` | Additive | Extreme disruption diagnostics. |
| DELAY_RATE | `SUM(CASE WHEN DEP_DELAY > 15 THEN 1 ELSE 0 END) / COUNT(*)` | Semi-additive | Main disruption indicator. |
| CANCELLATION_RATE | `SUM(CASE WHEN CANCELLATION_REASON <> 'Not Cancelled' THEN 1 ELSE 0 END) / COUNT(*)` | Semi-additive | Cancellation pressure by time band and context. |
| AVG_TAXI_OUT | `AVG(TAXI_OUT)` | Semi-additive | Ground operation pressure. |
| AVG_AIR_TIME | `AVG(AIR_TIME)` | Semi-additive | Airborne duration context. |
| AVG_DISTANCE | `AVG(DISTANCE)` | Semi-additive | Route-distance context. |
| AVG_WIND_SPD | `AVG(WIND_SPD)` | Non-additive | Weather context. |
| MAX_WIND_GUST | `MAX(WIND_GUST)` | Non-additive | Extreme wind context. |
| AVG_VISIBILITY | `AVG(VISIBILITY)` | Non-additive | Visibility context without over-emphasizing isolated zero readings. |
| LOW_VISIBILITY_RATE | `SUM(CASE WHEN VISIBILITY > 0 AND VISIBILITY <= 3 THEN 1 ELSE 0 END) / COUNT(*)` | Non-additive | Low-visibility scenario analysis. |

Table 3 defines the measures that can be used later in Tableau. It also prevents misleading aggregations: for example, weather readings are not summed, and rates must be recomputed at the selected level of detail.

## Logical Data Warehouse Design

The final logical schema follows the design in `phase 1 - design` and is summarized in Figure 6 and Table 4. The fact table is `FLIGHTS`. The dimensions are `DIM_FL_DATES`, `DIM_DEP_HOURS`, `DIM_STATIONS`, `DIM_CARRIERS`, `DIM_AIRCRAFTS`, and `DIM_JUNK`. The `DIM` prefix is preserved across SQL, notebooks, and generated warehouse CSV files.

`FLIGHTS` stores one row per departure. It contains surrogate foreign keys to all dimensions and the operational and meteorological measures needed for analysis. The table also includes `LOAD_TIMESTAMP`, a physical ETL metadata column that records when the warehouse batch was generated. `DIM_STATIONS` is reused for origin and destination airport roles, while `DIM_CARRIERS` is reused for marketing and operating carrier roles. This avoids duplicating the same descriptive entities. `DIM_JUNK` groups low-cardinality indicators such as cancellation status, active weather status, and route type.

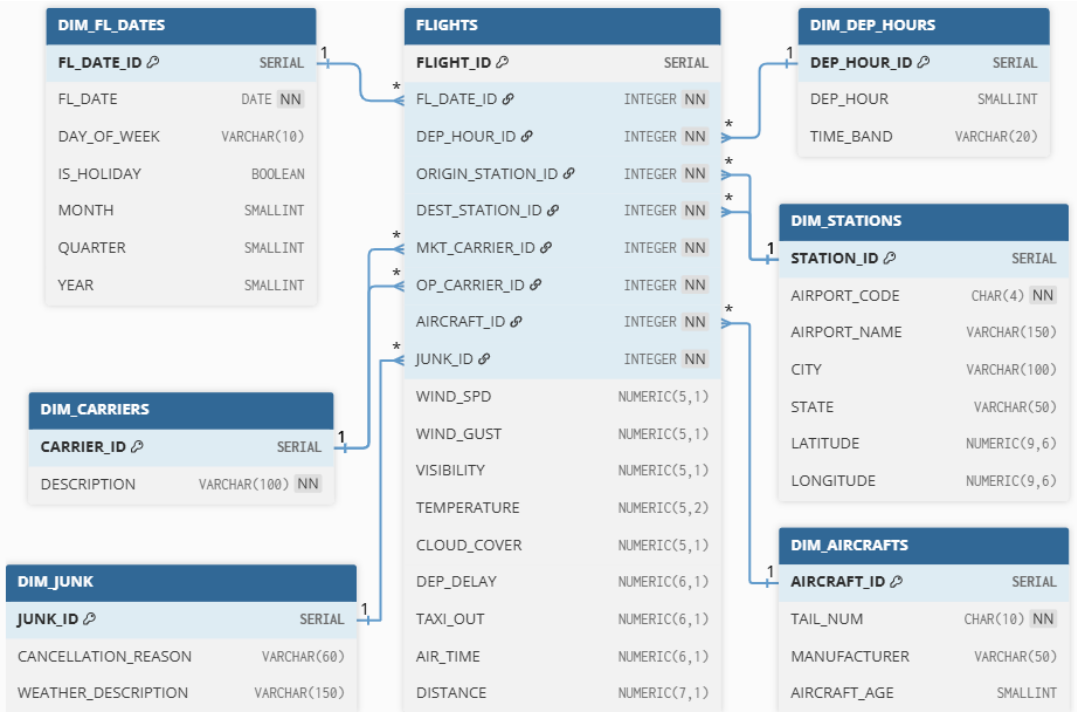


Figure 6: Logical star schema derived from the DFM.

Figure 6 shows the final relational form of the dimensional model. It makes explicit which surrogate keys connect the fact table to each dimension and which dimensions

are reused in more than one analytical role.

| Table | Primary key | Rows | Role |
|---|---|---|---|
| DIM_FL_DATES | FL_DATE_ID | 365 | Calendar dimension. |
| DIM_DEP_HOURS | DEP_HOUR_ID | 24 | Departure-hour and time-band dimension. |
| DIM_STATIONS | STATION_ID | 376 | Role-playing airport/station dimension. |
| DIM_CARRIERS | CARRIER_ID | 21 | Role-playing carrier dimension. |
| DIM_AIRCRAFTS | AIRCRAFT_ID | 6,129 | Aircraft registration and age dimension. |
| DIM_JUNK | JUNK_ID | 14 | Low-cardinality operational descriptors. |
| FLIGHTS | FLIGHT_ID | 6,921,847 | Departure fact table. |

Table 4: Final star schema tables and row counts generated by the ETL pipeline.

Table 4 reports the row counts produced by the final ETL run. These counts are used as a consistency check between the generated CSV files in `data/4-data warehouse/`, the ETL metadata in `output/6-ETL/`, and the schema described in this report.

## ETL Design and Implementation

The ETL pipeline is implemented in `phase 2 - etl/9-ETL Pipeline.ipynb`. It extracts the cleaned CSV files from `data/3-cleaned database/`, builds surrogate keys for each dimension, resolves role-playing foreign keys, and writes the final dimensional tables to `data/4-data warehouse/`. The implementation is notebook-based and uses reusable Python utilities for configuration, loading, and validation.

Dimensions are loaded first from the cleaned reconciled tables. The fact-loading step then joins cleaned flight records to dimension lookup tables and maps weather measures from the cleaned weather observations. This order allows all foreign keys in `FLIGHTS` to be checked against existing dimensions before the output is considered valid. The generated warehouse contains 6,921,847 rows in `FLIGHTS`. All fact foreign keys are populated, all expected `DIM` tables are present, all fact measures are non-null after the final cleaning pass, and `LOAD_TIMESTAMP` has one batch value per generated table. These checks confirm that the ETL output respects the logical schema defined in Phase 1.

## Conclusions

This project produced a complete data warehouse workflow for US domestic flight departures enriched with weather, carrier, aircraft, and station context. The main

result is not only the final set of dimensional CSV files, but the documented path from a raw denormalized source to a cleaned and validated star schema.

The most relevant design decisions were the deduplication and isolation of weather observations, the preservation of meaningful delay extremes, the use of role-playing dimensions for stations and carriers, and the alignment between the Phase 1 design and Phase 2 ETL outputs. The project also demonstrated why data quality assessment must precede dimensional modeling: without the DQA phase, duplicated weather observations, physically inconsistent meteorological values, and misleading outliers would have entered the warehouse silently.

## References

[1] JL8771, *2022 US airlines domestic departure data*, 2022. Accessed: May 1, 2025. [Online]. Available: https://www.kaggle.com/datasets/jl8771/2022-us-airlines-domestic-departure-data

[2] Bureau of Transportation Statistics, *Data dictionary and glossary: Airline on-time performance data*. Accessed: May 23, 2026. [Online]. Available: https://transtats.bts.gov/printglossary.asp

[3] Bureau of Transportation Statistics, *Reporting carrier on-time performance data table*. Accessed: May 23, 2026. [Online]. Available: https://www.transtats.bts.gov/TableInfo.asp?QO_fu146_anzr=b0-gvzr&gnoyr_VQ=FGJ

[4] Bureau of Transportation Statistics, *Technical directive: On-time reporting, effective january 1, 2019*, 2018. Accessed: May 23, 2026. [Online]. Available: https://www.bts.gov/topics/airlines-and-airports/number-31-technical-directive-time-reporting-effective-jan-1-2019

[5] Federal Aviation Administration, *Aircraft registration inquiry*. Accessed: May 23, 2026. [Online]. Available: https://registry.faa.gov/aircraftinquiry/

[6] National Weather Service, *ASOS visibility sensor documentation*. Accessed: May 23, 2026. [Online]. Available: https://www.weather.gov/asos/Visibility.html

[7] Iowa Environmental Mesonet, *ASOS/AWOS/METAR data download archive*. Accessed: May 23, 2026. [Online]. Available: https://mesonet.agron.iastate.edu/request/download.phtml

## Data Dictionary

This appendix provides the source-level data dictionary used during the preliminary analysis. The descriptions are based primarily on the Kaggle dataset documentation [1], with additional aviation and weather references where necessary. The tables are grouped by original CSV file so that each attribute can be traced back to the raw source. Each table lists the column name, the inferred type observed during profiling, and the meaning used in the design and ETL phases.

`CompleteData.csv`

| Column | Type | Description |
|---|---|---|
| FL_DATE | object | Date of flight |
| DEP_HOUR | int64 | Departure hour (0–23) |
| MKT_UNIQUE_ CARRIER | object | Marketing carrier unique code |
| MKT_CARRIER_ FL_NUM | int64 | Marketing carrier flight number |
| OP_UNIQUE_ CARRIER | object | Operating carrier unique code |
| OP_CARRIER_ FL_NUM | int64 | Operating carrier flight number |
| TAIL_NUM | object | FAA aircraft registration number |
| ORIGIN | object | Origin airport code |
| DEST | object | Destination airport code |
| DEP_TIME | object | Actual departure timestamp |
| CRS_DEP_TIME | object | Scheduled departure timestamp |
| TAXI_OUT | int64 | Taxi-out duration in minutes |
| DEP_DELAY | int64 | Departure delay in minutes |
| AIR_TIME | int64 | Flight air time in minutes |
| DISTANCE | int64 | Flight distance in miles |
| CANCELLED | int64 | Cancellation status code |
| LATITUDE | float64 | Airport latitude coordinate |
| LONGITUDE | float64 | Airport longitude coordinate |
| ELEVATION | int64 | Airport elevation in feet |

Table 5: Attribute description for the primary source dataset `CompleteData.csv`.

| Column | Type | Description |
|---|---|---|
| MESONET_ STATION | object | Mesonet weather station identifier |
| YEAR_OF_ MANUFACTURE | int64 | Aircraft manufacturing year |
| MANUFACTURER | object | Aircraft manufacturer name |
| ICAO_TYPE | object | ICAO aircraft type designator |
| RANGE | object | Aircraft range category |
| WIDTH | object | Aircraft body width category |
| WIND_DIR | float64 | Wind direction in degrees |
| WIND_SPD | float64 | Wind speed in knots |
| WIND_GUST | float64 | Wind gust speed in knots |
| VISIBILITY | float64 | Visibility in miles |
| TEMPERATURE | float64 | Temperature in Celsius |
| DEW_POINT | float64 | Dew point temperature in Celsius |
| REL_HUMIDITY | float64 | Relative humidity percentage |
| ALTIMETER | float64 | Altimeter setting in inches of mercury |
| LOWEST_ CLOUD_LAYER | float64 | Height of the lowest cloud layer in feet |
| N_CLOUD_ LAYER | float64 | Number of cloud layers detected |
| LOW_LEVEL_ CLOUD | float64 | Presence of low-level cloud formations |
| MID_LEVEL_ CLOUD | float64 | Presence of mid-level cloud formations |
| HIGH_LEVEL_ CLOUD | float64 | Presence of high-level cloud formations |
| CLOUD_COVER | float64 | Cloud cover measurement |
| ACTIVE_ WEATHER | float64 | Encoded active weather condition |

Table 5: Attribute description for the primary source dataset `CompleteData.csv` (continued).

**ActiveWeather.csv**

| Column | Type | Description |
|---|---|---|
| STATUS | int64 | Primary key of the weather status |
| WEATHER_ DESCRIPTION | object | Description of the active weather condition |

Table 6: Attribute description for ActiveWeather.csv.

**Cancellation.csv**

| Column | Type | Description |
|---|---|---|
| STATUS | int64 | Primary key of the cancellation status |
| CANCELLATION_ REASON | object | Semantic description of the cancellation reason |

Table 7: Attribute description for Cancellation.csv.

**Carriers.csv**

| Column | Type | Description |
|---|---|---|
| CODE | object | Unique carrier identifier |
| DESCRIPTION | object | Full airline carrier description |

Table 8: Attribute description for Carriers.csv.

**Stations.csv**

| Column | Type | Description |
|---|---|---|
| AIRPORT_ID | int64 | Internal numeric airport identifier |
| AIRPORT | object | Airport code |
| DISPLAY_ AIRPORT_NAME | object | Full airport name |
| DISPLAY_ AIRPORT_CITY_ NAME_FULL | object | Full airport city name |
| AIRPORT_ STATE_NAME | object | Full airport state name |
| AIRPORT_ STATE_CODE | object | Airport state abbreviation |
| LATITUDE | float64 | Latitude coordinate |
| LONGITUDE | float64 | Longitude coordinate |
| ELEVATION | int64 | Airport elevation in feet |
| ICAO | object | ICAO airport identifier |
| IATA | object | IATA airport identifier |
| FAA | object | FAA airport identifier |
| MESONET_ STATION | object | Associated Mesonet station identifier |

Table 9: Attribute description for Stations.csv.